Enhanced LLM-Aided Testbench Generation Report

Automatic Verilog testbench generation with golden reference outputs

Course: LLM4ChipDesign (Spring 2026)

Name: Aviraj Dongare

1. Introduction

Hardware verification is one of the most critical stages in digital system design. Writing comprehensive testbenches manually can be time-consuming and error-prone. The goal of this assignment was to explore how **Large Language Models (LLMs)** can assist in automatically generating testbenches for Verilog hardware modules.

In this project, an **LLM-aided verification pipeline** was implemented that automates the process of:

1. Understanding a hardware module from **natural language description and Verilog code**
2. Generating **test patterns**
3. Producing a **Python golden reference model**
4. Computing **expected outputs**
5. Generating a **self-checking Verilog testbench**

The system integrates **LLM reasoning with traditional hardware simulation tools** such as **Icarus Verilog (iverilog)** and **vvp**.

The assignment was completed in four main parts:

- **Part I:** Running tutorial examples
- **Part II:** Understanding generated artifacts
- **Part III:** Running a custom module through the pipeline
- **Part IV:** Demonstrating bug detection capability

The following sections summarize the experiments, results, and verification process.

2. LLM Setup and Configuration

The pipeline uses a Large Language Model to interpret hardware specifications and generate verification artifacts. The LLM was accessed using the **Anthropic API**.

Model Used

The model used for the assignment was:

Claude-3.5-Haiku

This model was selected because it provides a good balance between reasoning capability and response latency, which is suitable for automated generation of verification artifacts.

API Configuration

The API key was stored securely in the notebook environment using environment variables.

Example initialization used in the notebook:

```
pipeline = TestbenchPipeline(
    api_key=os.environ["ANTHROPIC_API_KEY"],
    model=model_choice,
    provider="anthropic"
)
```

Key Parameters

Important parameters used in the pipeline:

Parameter	Description
model	LLM model used for generation
description	Natural language hardware specification
verilog_code	DUT implementation
output_dir	Directory where generated artifacts are stored

Prompting Strategy

The prompt given to the LLM contains:

1. **Natural language description of the module**
2. **Verilog DUT implementation**
3. Instructions to generate:
    - test patterns
    - golden reference model
    - verification logic

The LLM interprets the hardware functionality and generates verification artifacts consistent with the specification.

3. LLM-Aided Testbench Generation Pipeline

The notebook implements a **5-step pipeline** that automates the verification process.

Step 1 – Input Processing

The pipeline receives two inputs:

- Natural language description of the hardware
- Verilog code of the DUT

These inputs allow the LLM to understand both **functional intent** and **implementation details**.

Step 2 – Test Pattern Generation

The LLM generates multiple test patterns covering:

- normal functional cases
- corner cases
- edge conditions
- randomized inputs

These patterns are used to stimulate the hardware module during simulation.

Step 3 – Initial Testbench Generation

The pipeline generates an initial Verilog testbench containing:

- DUT instantiation
- signal declarations
- stimulus logic

The testbench applies the generated test patterns to the DUT inputs.

The generated file:

```
testbench_initial.v
```

Step 4 – Golden Model Generation

The LLM generates a **Python golden reference model** that represents the correct functionality of the hardware module.

This golden model is used to compute the expected outputs for each test pattern.

Generated file:

```
golden_model.py
```

Step 5 – Final Self-Checking Testbench

The pipeline combines the test patterns and golden outputs to create a **self-checking testbench**.

This testbench automatically compares DUT outputs with the expected results.

Generated file:

```
testbench_final.v
```

If mismatches occur, the testbench reports them during simulation.

4. Tutorial Examples Executed

Two tutorial examples were executed to demonstrate the pipeline functionality.

Example 1 – 2-to-1 Multiplexer

Module Description

The first example was a **2-to-1 multiplexer**.

Inputs:

```
a : 1-bit input
b : 1-bit input
sel : select signal
```

Output:

```
y : selected input
```

DUT Interface

```
module mux2to1(
input wire a,
input wire b,
input wire sel,
output wire y
);
```

Functionality

The multiplexer outputs:

```
y = sel ? b : a
```

If `sel` is 0, input `a` is selected.
If `sel` is 1, input `b` is selected.

Simulation Commands Used

Compilation:

```
iverilog -g2012 -o simv mux.v testbench_final.v
```

Simulation:

```
vvp simv
```

Result

All generated test patterns passed successfully.

The testbench correctly verified the functionality of the multiplexer module.

Example 2 – 4-bit Adder

The second tutorial example implemented a **4-bit adder with carry output**.

DUT Interface

```
module adder4bit(
input [3:0] a,
input [3:0] b,
output [3:0] sum,
output carry
);
```

Functionality

The module performs:

```
sum = a + b
carry = carry-out
```

The lower four bits represent the sum while the carry flag indicates overflow.

Simulation Commands

Compilation:

```
iverilog -g2012 -o simv adder4bit.v testbench_final.v
```

Simulation:

```
vvp simv
```

Result

The generated testbench verified multiple combinations of inputs and confirmed that the adder implementation behaved correctly.

5. Explanation of Generated Artifacts

Two important artifacts are generated by the pipeline.

golden_model.py

The file **golden_model.py** represents a **software reference implementation** of the hardware module.

It computes the correct output values based on the module specification.

Purpose:

- Serves as a correctness reference
- Used to compute expected outputs for each test pattern
- Ensures verification is independent of DUT implementation

Example behavior for an adder:

```
sum = a + b
carry = (a + b) > 15
```

The golden model is executed during the pipeline to generate correct outputs for all test cases.

test_patterns_with_golden.json

This file stores all generated test cases.

Each test entry contains:

```
{
 "inputs": {...},
 "expected_outputs": {...}
}
```

Example structure:

```
{
 "req": 6,
 "expected_outputs": {
  "code": 2,
  "valid": 1
 }
}
```

Purpose:

- stores input vectors
- stores expected outputs from the golden model

- allows the final testbench to verify DUT correctness

This file acts as a bridge between **Python golden reference verification** and **Verilog simulation**.

6. Custom Module – 4-bit Priority Encoder

In Part III, a custom module was created and verified using the LLM pipeline.

Module Description

The module implements a **4-bit priority encoder**.

Inputs:

```
req[3:0] : request vector
```

Outputs:

```
code[1:0] : index of highest priority request
valid : indicates at least one request is active
```

Priority order:

```
req[3] highest
req[2]
req[1]
req[0] lowest
```

Encoding Rules

Input	Output
1000	11
0100	10
0010	01
0001	00

If multiple bits are active, the highest priority bit is encoded.

Example:

```
req = 1011 → code = 11
```

DUT Interface

```
module priority_encoder4(
input [3:0] req,
output reg [1:0] code,
output reg valid
);
```

Simulation Commands Used

Compilation:

```
iverilog -g2012 -o simv priority_encoder4.v testbench_final.v
```

Simulation:

```
vvp simv
```

Result

The pipeline generated **25 test patterns**, exceeding the minimum requirement of 20.

All tests passed successfully, confirming correct implementation of the priority encoder.

7. Bug Detection Demonstration

Part IV required demonstrating that the generated testbench can detect implementation errors.

Intentional Bug Introduction

A small bug was intentionally introduced into the DUT.

Original implementation:

```
4'b01xx: code = 2'b10;
```

Buggy implementation:

```
4'b01xx: code = 2'b01;
```

This causes incorrect encoding when `req[2]` is the highest priority request.

Failing Simulation Result

After compiling the buggy DUT and running the testbench, multiple test cases failed.

Example failing outputs:

```
Test case 4: req = 0100 FAIL
Test case 7: req = 0111 FAIL
Test case 15: req = 0110 FAIL
```

Final summary:

```
Total tests run: 25
Passed: 20
Failed: 5
```

This confirms that the generated testbench correctly detected the implementation error.

Bug Fix and Verification

The DUT was then corrected by restoring the correct encoding rule.

Correct implementation:

```
4'b01xx: code = 2'b10;
```

After recompiling and rerunning the simulation:

```
Total tests run: 25
Passed: 25
Failed: 0
```

This confirms that the module now behaves correctly and that the testbench accurately verifies the design.

8. Conclusion

This assignment demonstrated the effectiveness of **LLM-aided verification for hardware design**.

Key achievements include:

- Automated generation of testbenches from natural language descriptions
- Creation of golden reference models
- Generation of comprehensive test patterns
- Automated verification using self-checking testbenches
- Successful detection of implementation bugs

The pipeline significantly reduces manual verification effort while improving test coverage.

The results show that combining **LLM reasoning with traditional hardware simulation tools** can greatly improve productivity in digital design verification.